

1. Microservices - JWT

Question 1:

Implement JWT Authentication in ASP.NET Core Web API

Solution:

Install NuGet Packages:

dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
appsettings.json:

```
{  
  "Jwt": {  
    "Key": "ThisIsASecretKeyForJwtToken",  
    "Issuer": "MyAuthServer",  
    "Audience": "MyApiUsers",  
    "DurationInMinutes": 60  
  }  
}
```

Program.cs:

```
builder.Services.AddAuthentication("Bearer")  
    .AddJwtBearer("Bearer", options =>  
    {  
        options.TokenValidationParameters = new TokenValidationParameters  
        {  
            ValidateIssuer = true,  
            ValidateAudience = true,  
            ValidateLifetime = true,  
            ValidateIssuerSigningKey = true,  
            ValidIssuer = builder.Configuration["Jwt:Issuer"],  
            ValidAudience = builder.Configuration["Jwt:Audience"],  
            IssuerSigningKey = new  
            SymmetricSecurityKey(Encoding.UTF8.GetBytes(builder.Configuration["Jwt  
:Key"]))  
        };  
    });  
builder.Services.AddAuthorization();
```

AuthController.cs:

```
[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    [HttpPost("login")]
    public IActionResult Login([FromBody] LoginModel model)
    {
        if (IsValidUser(model))
        {
            var token = GenerateJwtToken(model.Username);
            return Ok(new { Token = token });
        }
        return Unauthorized();
    }
    {
        private string GenerateJwtToken(string username)
        {
            var claims = new[]
            {
                new Claim(ClaimTypes.Name, username)
            };
            var key = new
                SymmetricSecurityKey(Encoding.UTF8.GetBytes("ThisIsASecretKeyForJwtToken"));
            var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
            var token = new JwtSecurityToken(
                issuer: "MyAuthServer",
                audience: "MyApiUsers", claims: claims,
                expires: DateTime.Now.AddMinutes(60),
                signingCredentials: creds);
            return new JwtSecurityTokenHandler().WriteToken(token);
        }
    }
}
```

```

CleanJwtDemo — CleanJwtDemo - dotnet run — 80x37
...Demo — CleanJwtDemo - dotnet run  ~/CleanJwtDemo — -zsh +

app.MapGet("/api/auth/secure-data", [Authorize] () =>
{
    return Results.Ok(new { message = "Access granted! Secured by JWT Minimal AP
I." });
});

app.Run();

public class LoginModel
{
    public string Username { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
}
EOF
(base) chapalasuryatej@Chapalas-MacBook-Air CleanJwtDemo % dotnet run
Using launch settings from /Users/chapalasuryatej/CleanJwtDemo/Properties/launch
Settings.json...
Building...
Restore succeeded with 1 warning(s) in 0.3s
/Users/chapalasuryatej/CleanJwtDemo/CleanJwtDemo.csproj : warning NU1903: Pa
ckage 'Microsoft.OpenApi' 2.0.0 has a known high severity vulnerability, https:/
/github.com/advisories/GHSA-v5pm-xwqc-g5wc
CleanJwtDemo net10.0 succeeded with 1 warning(s) (1.5s) → bin/Debug/net10.0/Cl
eanJwtDemo.dll
/Users/chapalasuryatej/CleanJwtDemo/CleanJwtDemo.csproj : warning NU1903: Pa
ckage 'Microsoft.OpenApi' 2.0.0 has a known high severity vulnerability, https:/
/github.com/advisories/GHSA-v5pm-xwqc-g5wc
info: Microsoft.Hosting.Lifetime[14]
Now listening on: http://localhost:5091
info: Microsoft.Hosting.Lifetime[0]
Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
Content root path: /Users/chapalasuryatej/CleanJwtDemo

```

```
CleanJwtDemo — -zsh — 80x37
...Demo — CleanJwtDemo - dotnet run      ~/CleanJwtDemo — -zsh    +

Last login: Sat Jul 18 17:15:30 on ttys000
(base) chapalasuryatej@Chapalas-MacBook-Air CleanJwtDemo % curl -X POST http://localhost:5091/api/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"admin", "password":"password123"}'
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZWl1cy54bWxzbnZfLm9yZy93cy8yMDAxLzA1L2lkZW50aXR5L2NsYWltcy9uYVllIjoieWRTaW4iLCJqdGkiOiJlN2Y0ZmNiYi1hNzEzLTlMTetOWQxOS1jZGE3NmIwZjEwZTtkIlk1eHAiOjE3ODQzNzg0OTIsImVzcyI6Ikd1ZXV0aFNlcnciOiJmF1ZCI6Ik15QXBvVXNlcnMifQ.xILrkav11KJOLSij1zWoHFe5KbsWcrhRJIP37uV2JM0"}%
(base) chapalasuryatej@Chapalas-MacBook-Air CleanJwtDemo % curl -i http://localhost:5091/api/auth/secure-data \
-H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZWl1cy54bWxzbnZfLm9yZy93cy8yMDAxLzA1L2lkZW50aXR5L2NsYWltcy9uYVllIjoieWRTaW4iLCJqdGkiOiJlN2Y0ZmNiYi1hNzEzLTlMTetOWQxOS1jZGE3NmIwZjEwZTtkIlk1eHAiOjE3ODQzNzg0OTIsImVzcyI6Ikd1ZXV0aFNlcnciOiJmF1ZCI6Ik15QXBvVXNlcnMifQ.xILrkav11KJOLSij1zWoHFe5KbsWcrhRJIP37uV2JM0"
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Date: Sat, 18 Jul 2026 11:48:57 GMT
Server: Kestrel
Transfer-Encoding: chunked

{"message":"Access granted! Secured by JWT Minimal API."}%
(base) chapalasuryatej@Chapalas-MacBook-Air CleanJwtDemo %
```